Keeping data private
in real-time pipelines

by Olena Kutsenko,
Confluent, an IBM Company



Don’t expose them to bright light.

Don’t get them wet.

Never feed them after midnight.





Don’t expose them to bright light.

Don’t get them wet.

Never feed them after midnight.





Privacy issues



Data



```
03:14:08 INFO  login request received
03:14:08 DEBUG payload.email=gremlin@example.com
03:14:08 DEBUG payload.device=ios
03:14:08 DEBUG payload.remember_me=true
03:14:08 DEBUG payload.password=MyPrecious!!!
03:14:08 INFO  login successful
```



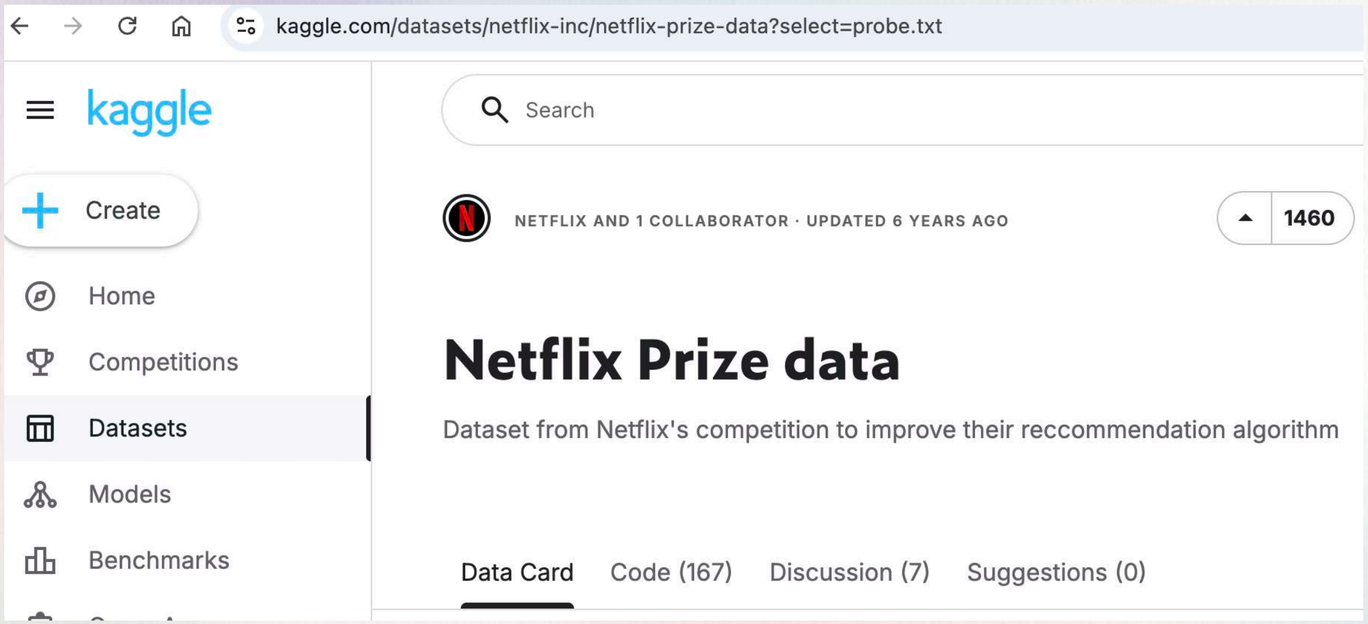


Anonymous == safe ?





Netflix Prize Dataset





- a training data set of **100,480,507 ratings**

that **480,189 users** gave to **17,770 movies**

For each movie:
- CustomerID
- Rating
- Date

Robust De-anonymization of Large Datasets (How to Break Anonymity of the Netflix Prize Dataset)

Arvind Narayanan and Vitaly Shmatikov

The University of Texas at Austin

November 26, 2024

arXiv:cs/0610105v2 [cs.CR] 22 Nov 2007

Abstract

We present a new class of statistical de-anonymization attacks against high-dimensional micro-data, such as individual preferences, recommendations, transaction records and so on. Our techniques are robust to perturbation in the data and tolerate some mistakes in the adversary's background knowledge.

We apply our de-anonymization methodology to the Netflix Prize dataset, which contains anonymous movie ratings of 500,000 subscribers of Netflix, the world's largest online movie rental service. We demonstrate that an adversary who knows only a little bit about an individual subscriber can easily identify this subscriber's record in the dataset. Using the Internet Movie Database as the source of background knowledge, we successfully identified the Netflix records of known users, uncovering their apparent political preferences and other potentially sensitive information.

1 Introduction

Robust De-anonymization of Large Datasets (How to Break Anonymity of the Netflix Prize Dataset)

Arvind Narayanan and Vitaly Shmatikov

The University of Texas at Austin

November 26, 2024

arXiv:cs/0610105v2 [cs.CR] 22 Nov 2007

Abstract

We present a new class of statistical de-anonymization attacks against high-dimensional micro-data, such as individual preferences, recommendations, transaction records and so on. Our techniques are robust to perturbation in the data and tolerate some mistakes in the adversary's background knowledge.

We apply our de-anonymization methodology to the Netflix Prize dataset, which contains anonymous movie ratings of 500,000 subscribers of Netflix, the world's largest online movie rental service. We demonstrate that an adversary who knows only a little bit about an individual subscriber can easily identify this subscriber's record in the dataset. Using the Internet Movie Database as the source of background knowledge, we successfully identified the Netflix records of known users, uncovering their apparent political preferences and other potentially sensitive information.

1 Introduction


Flink
CONFLUENT
Developer
ICEBERG
kafka

Privacy nightmare in streaming systems



- Always on
- Low latency
- Automated
- No human checkpoint

produce

message

kafka

topic

copied

consumed

Java

MongoDB.

processed

enriched



Fan-out: one event, many destinations

- Always on
- Low latency
- Automated
- No human checkpoint

produce

kafka

topic

message

copied

consumed

Java

MongoDB.

processed

enriched



Fan-out: one event, many destinations

- Always on
- Low latency
- Automated
- No human checkpoint

produce

kafka

topic

message

copied

consumed

Java

MongoDB.

processed

enriched



Level 1: foundation



Foundation



Identity & access control

- Authorization
- Authentication
- Role-based access control



Foundation



Identity & access control

- Authorization
- Authentication
- Role-based access control

Data governance & policy

- Schema Registry/data contracts
- PII tags
- Validation or transformation rules
- Lineage and catalog visibility



Foundation



Identity & access control

- Authorization
- Authentication
- Role-based access control

Encryption

- At rest
- In transit
- Client-side field level encryption (CSFLE)
- Client-side payload encryption (CSPE)

Data governance & policy

- Schema Registry/data contracts
- PII tags
- Validation or transformation rules
- Lineage and catalog visibility



Foundation



Identity & access control

- Authorization
- Authentication
- Role-based access control

Encryption

- At rest
- In transit
- Client-side field level encryption (CSFLE)
- Client-side payload encryption (CSPE)

Data governance & policy

- Schema Registry/data contracts
- PII tags
- Validation or transformation rules
- Lineage and catalog visibility

Audit logs and evidence for compliance

- Track user and application access
- Identify abnormal behavior and anomalies
- Meet regulatory compliance requirements
- Investigate security incidents and policy violations
- Monitor resource usage and operational changes



Level 2: data masking & tokenization



Data masking



- john.doe@gmail.com → j***@gmail.com
- Credit card → **** **** **** 1234
- Phone → +1-***-***-4421



Core principles

- Often retains the structure, format, and logic of the original
- Should prevent tracing back to the original values
- Be consistent when needed

Common masking techniques

- Substitution
- Shuffling
- Number/date variance
- Encryption
- Nulling out / deletion
- Masking out

Different types of data masking

- Static vs dynamic
- Deterministic or random
- Statistical or record-level
- On-the-fly or pre-computed

```
CREATE TABLE customers_source
(
    customer_id BIGINT,
    name        STRING,
    email       STRING,
    address     STRING,
    postcode    STRING,
    city        STRING,
    monthly_spend DOUBLE,
    signup_date DATE
);
```



CONFLUENT

Search

workspace-2025-03-03-080... | workspace-2025-02-18-0756... | workspace-2025-12-22-2037...

workspace-2025-12-22-203759

olena-env

Olena Kutsenko

User Account

AWS | us-east-1

AWS.us-east-1.env-j0jn2m.1ea6 | Running

Catalog olena-env Database cluster_0

Search...

customer_id	name	email	address	postcode	city
1	Alice Müller	alice.mueller@exampl...	Alexanderplatz 1	10178	Berlin
2	Bob Schneider	bob.schneider@exam...	Leopoldstrasse 50	80802	Munich
3	Carla Fischer	carla.fischer@example...	Reeperbahn 7	20359	Hamburg
4	David Weber	david.weber@example...	Zeil 10	60313	Frankfurt
5	Emily Clark	emily.clark@example.c...	10 Downing Street	SW1A 2AA	London
6	François Dupont	francois.dupont@mail.fr	5 Rue de Rivoli	75001	Paris
7	George Miller	george.miller@corp.us	600 Market Street	94105	San Francisco
8	Hannah Johnson	hannah.johnson@ny-r...	350 5th Avenue	10001	New York
9	Isha Verma	isha.verma@shop.in	MG Road 12	560001	Bengaluru

Last saved at 2/9/26, 10:02 AM

7 / 20 cells

```sql
SELECT customer_id,
       -- Full mask: completely hide the value
       '***MASKED***'                                    AS name_masked_full,

       -- Partial mask: keep first letter, mask the rest
       CONCAT(SUBSTRING(name, 1, 1), '***')              AS name_masked_partial,

       -- Email masking: hide user part, keep domain
       CONCAT('***@', SUBSTRING(email, POSITION('@' IN email) + 1))
                                                         AS email_masked_domain_only,
       -- Keep schema but drop value
       CAST(NULL AS STRING)                              AS name_null,
       CAST(NULL AS STRING)                              AS email_null,
       CAST(NULL AS STRING)                              AS address_null,

       -- Replace with plausible but fake values
       CASE city
           WHEN 'Berlin' THEN 'City-A'
           WHEN 'Munich' THEN 'City-B'
           WHEN 'Hamburg' THEN 'City-C'
           WHEN 'Frankfurt' THEN 'City-D'
           ELSE 'City-X'
           END                                           AS city_substituted,

       -- Substitute postcode pattern but keep rough region information
       CONCAT(SUBSTRING(postcode, 1, 1), '0000') AS postcode_substituted

  FROM customers_source;
```



CONFLUENT

Search

workspace-2025-03-03-080... | workspace-2025-02-18-0756... | workspace-2025-12-22-2037...

workspace-2025-12-22-203759

olena-env

Olena Kutsenko

User Account

AWS | us-east-1

AWS.us-east-1.env-j0jn2m.1ea6 | Running

Catalog olena-env Database cluster_0

customer_id	name_masked_full	name_masked_pa...	email_masked_do...	name_null	email_
10	***MASKED***	J***	***@loja.br	NULL	NULL
11	***MASKED***	K***	***@company.jp	NULL	NULL
12	***MASKED***	L***	***@example.com.au	NULL	NULL
13	***MASKED***	M***	***@example.ca	NULL	NULL
14	***MASKED***	N***	***@example.nl	NULL	NULL
15	***MASKED***	O***	***@example.es	NULL	NULL
16	***MASKED***	P***	***@example.it	NULL	NULL
17	***MASKED***	S***	***@example.se	NULL	NULL
18	***MASKED***	T***	***@example.pl	NULL	NULL
19	***MASKED***	Z***	***@example.za	NULL	NULL
20	***MASKED***	W***	***@example.sg	NULL	NULL
COUNT 20	COUNT 20	COUNT 20	COUNT 20	COUNT 0	COUN

Last saved at 2/9/26, 10:02 AM

7 / 20 cells

Data masking



- john.doe@gmail.com → j***@gmail.com
- Credit card → **** **** **** 1234
- Phone → +1-***-***-4421



- Governed masking: tag fields as PII and apply a Schema Registry rule
- No-code masking: Flink Mask Fields in the Cloud UI
- Connector-edge masking: single message transforms

```
{
  "type":"record",
  "name":"MyRecord",
  "fields":[{
    "name":"ssn",
    "type":"string",
    "confluent:tags": [ "PII", "PRIVATE" ]
  }]
}
```

```
"domainRules": [ {
  "name": "maskPII",
  "doc": "Mask all PII tagged fields",
  "kind": "TRANSFORM",
  "mode": "WRITE",
  "type": "CEL_FIELD",
  "tags": [ "PII" ],
  "expr": "\"XXXXX\"",
  "onSuccess": "NONE",
  "onFailure": "ERROR",
 "disabled": false } ]
```

Tokenization



Email → tok_8f3a91
User ID → u_492001



How it works

- Tokenization replaces real values with meaningless stand-ins
- Tokens have no meaning outside the system
- Mapping lives in a secure store or service

Types of tokens

- High-Value Tokens - can be used in place of the real data
- Low-Value Tokens - only a pointer
- Deterministic and random

Key components

- Token generation
- Token mapping
- Token vault (data store)
- Key management

Tokenization vs encryption

- Tokenization replaces data with a surrogate
- Encryption mathematically transforms data
- Hashing is not tokenization

```sql
SELECT
    customer_id,
    -- Assign cities randomly to break direct linkage
    CASE
        WHEN RAND() < 0.25 THEN 'Shuffle-City-1'
        WHEN RAND() < 0.50 THEN 'Shuffle-City-2'
        WHEN RAND() < 0.75 THEN 'Shuffle-City-3'
        ELSE 'Shuffle-City-4'
        END AS city_shuffled_demo

FROM customers_source;
```

```sql
SELECT c.customer_id,
       -- Low-value token: pointer; only the vault “knows” what it means
       v.city_token_low AS city_token_low

FROM customers_source AS c
         LEFT JOIN city_token_vault AS v
                   ON c.city = v.real_city;
```



CONFLUENT

Search

workspace-2025-03-03-080...

workspace-2025-02-18-0756...

workspace-2025-12-22-2037...

workspace-2025-12-22-203759

olena-env

Olena Kutsenko

User Account

AWS | us-east-1

AWS.us-east-1.env-j0jn2m.1ea6 | Running

Catalog olena-env

Database cluster_0

customer_id	city_shuffled_demo
1	Shuffle-City-2
2	Shuffle-City-2
3	Shuffle-City-1
4	Shuffle-City-4
5	Shuffle-City-1
6	Shuffle-City-3
7	Shuffle-City-2
8	Shuffle-City-1
9	Shuffle-City-2
10	Shuffle-City-3
11	Shuffle-City-1

Last saved at 2/9/26, 10:02 AM

7 / 20 cells

CONFLUENT

Search

workspace-2025-03-03-080... | workspace-2025-02-18-0756... | workspace-2025-12-22-2037...

workspace-2025-12-22-203759

olena-env

Olena Kutsenko

User Account

AWS | us-east-1

AWS.us-east-1.env-j0jn2m.1ea6 | Running

Catalog olena-env Database cluster_0

```
1  SELECT c.customer_id,
2         -- Low-value token: pointer; only the vault “knows” what it means
3         v.city_token_low AS city_token_low
4
5  FROM customers_source AS c
6           LEFT JOIN city_token_vault AS v
7                     ON c.city = v.real_city;
```

START TIME:	STATEMENT STATUS:	STATEMENT NAME:	RESULTS:
2026-02-09T08:59:47.307134Z	Running	workspace-2025-12-22-203759-4198aac3-7f46-4958-b57e-09e5b5d2e...	20

Stop

Search...

123 customer_id	Aa city_token_low
1	ctok_a91f23
2	ctok_7bc942
3	ctok_5d88ff

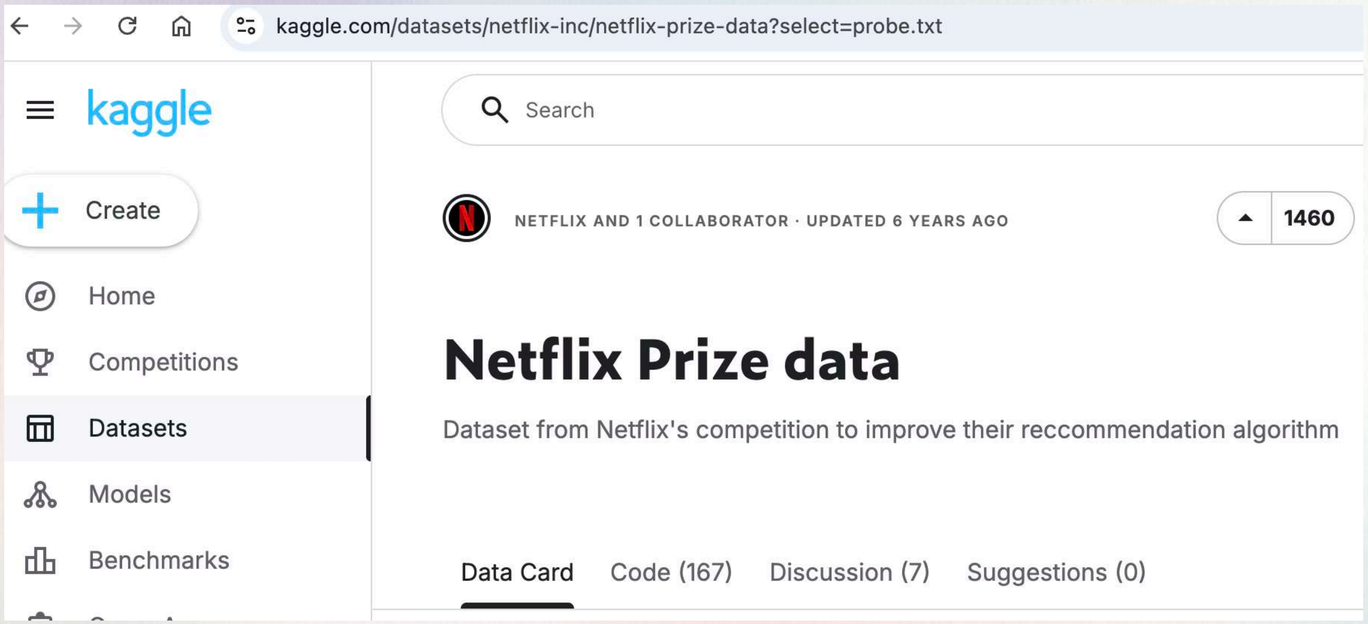
Last saved at 2/9/26, 10:02 AM

7 / 20 cells

Re-identification attacks



Netflix Prize Dataset





- a training data set of **100,480,507 ratings**

that **480,189 users** gave to **17,770 movies**

For each movie:
- CustomerID
- Rating
- Date

https://www.something awful.com/weekend-web/aol-search-log/5/

AOL search log release

- **twenty million** search queries for over **650,000 users** over **a three-month** period
- released on August 4th 2006, deleted on August 7th 2006

7979427	how do you talk to people with the microphone	2006-05-10 1
7979427	how do you talk to people with the microphone	2006-05-10 3
7979427	how do you talk to people with the microphone	2006-05-10 3
7979427	how do you talk to people with the microphone	2006-05-10 3
7979427	do i already have ya hoo messenger	2006-05-10 0
7979427	do i already have yahoo messenger	2006-05-10 0
7979427	do i already have yahoo messenger	2006-05-10 0
	i have been trying to find out my e	

CNN Money

101 DUMBEST MOMENTS
The year's biggest boors, buffoons, and blunderers

57 AOL: Search queries (TWX)

I Love Alaska is a 2009 documentary chronicling the AOL search history of "user 711391," whose searches are narrated by a monotone female voiceover.[1] The film was produced by Submarine Channel, and released episodically in 2009 before being uploaded to stream for free on Minimovies.org.[2]

Plot [edit]

Only glimpsing the life of user 711391 through her search history, we are introduced to a middle-aged woman from Texas looking to rejuvenate her sex life and dreaming of life in Alaska.[3]

1 0

3 1

4 0

Medical records

https://techscience.org/a/2015092903/

techscience.org/a/2015092903/

Only You, Your Doctor, and Many Others May Know

Latanya Sweeney

Record	[illegible]
Hospital	162: Sacred Heart Medical Center in Providence
Admit Type	1: Emergency
Type of Stay	[illegible]
Length of Stay	6 days
Discharge Date	Oct-2011
Discharge Status	[illegible] under the care of an health service organization
Charges	$71708.47
Payers	1: Medicare
	6: Commercial insurance
	625: Other government sponsored patients
Emergency Codes	E8162: motor vehicle traffic accident due t[illegible] loss of control; loss control mv-mocycl
Diagnosis	[illegible]

MAN, 60, THROWN FROM MOTORCYCLE

A 60-year-old Soap Lake man was hospitalized Saturday afternoon after he was thrown from his motorcycle. Ronald Jameson was riding his 2003 Harley-Davidson north on Highway 25, when he failed to negotiate a curve to the left. His motorcycle became airborne before landing in a wooded area. Jameson was thrown from the bike;

Strava tracking data

- every single activity ever uploaded to Strava – more than **3 trillion individual GPS data points**

Fitness tracking app Strava gives away location of secret US army bases

Data about exercise routes shared online by soldiers can be used to pinpoint overseas facilities

Latest: Strava suggests military users 'opt out' of heatmap as row deepens



Helmand province in Afghanistan. Photograph: Strava heatmap



RAF Mount Pleasant in the Falkland Islands. Photograph: Strava heatmap



NYC taxi data

- 173m individual taxi trips
- 20GB of uncompressed data

https://chriswhong.com/open-data/foil_nyc_taxi/

trip_data_1.csv

```
1  medallion, hack_license, vendor_id, rate_code, store_and_fwd_flag,
   pickup_datetime, dropoff_datetime, passenger_count, trip_time_in_secs,
   trip_distance, pickup_longitude, pickup_latitude, dropoff_longitude,
   dropoff_latitude
2  89D227B655E5C82AECF13C3F540D4CF4,BA96DE419E711691B9445D6A6307C170,CMT,1,N,2013-01-
   01 15:11:48,2013-01-01 15:18:10,4,382,1.
   00,-73.978165,40.757977,-73.989838,40.751171
3  0BD7C8F5BA12B88E0B67BED28BEA73D8,9FD8F69F0804BDB5549F40E9DA1BE472,CMT,1,N,2013-01-
   06 00:18:35,2013-01-06 00:22:54,1,259,1.50,-74.006683,40.731781,-73.994499,40.75066
4  0BD7C8F5BA12B88E0B67BED28BEA73D8,9FD8F69F0804BDB5549F40E9DA1BE472,CMT,1,N,2013-01-
   05 18:49:41,2013-01-05 18:54:23,1,282,1.10,-74.004707,40.73777,-74.009834,40.726002
5  DFD2202EE08F7A8DC9A57B02ACB81FE2,51EE87E3205C985EF8431D850C786310,CMT,1,N,2013-01-
   07 23:54:15,2013-01-07 23:58:20,2,244,.70,-73.974602,40.759945,-73.984734,40.759388
6  DFD2202EE08F7A8DC9A57B02ACB81FE2,51EE87E3205C985EF8431D850C786310,CMT,1,N,2013-01-
   07 23:25:03,2013-01-07 23:34:24,1,560,2.10,-73.97625,40.748528,-74.002586,40.747868
7  20D9ECB2CA0767CF7A01564DF2844A3E,598CCE5B9C1918568DEE71F43CF26CD2,CMT,1,N,2013-01-
   07 15:27:48,2013-01-07
   15:38:37,1,648,1.70,-73.966743,40.764252,-73.983322,40.743763
8  496644932DF3932605C22C7926FF0FE0,513189AD756FF14FE670D10B92FAF04C,CMT,1,N,2013-01-
```

Data doesn’t need names to identify people

Re-identification	Mapping data back to a real person
Deanonymization	Breaking an anonymity claim
Linkage attack	Joining with auxiliary datasets
Singling-out	Isolating one individual in a dataset
Inference attack	Learning hidden attributes
Reconstruction	Rebuilding sensitive data
Linkability	Tracking the same person across records
Semantic leakage	Identity revealed via meaning/content
Trajectory attack	Identifying people via movement



Level 3: advanced techniques



k-anonymity



For every record, there must be **at least k people** with the same visible attributes

Methods

- Suppression
- Generalization

Age: 27
ZIP: 12345
Gender: F
Religion: Hindu

→

Age: 25 - 30
ZIP: 123**
Gender: F
Religion: *

Bucketing



Bucketing replaces exact values with ranges.

Age → 20–29
Salary → 50k–75k
Location → city instead of GPS
Time → hour instead of second

```sql
SELECT
    customer_id,
    -- Annual spend (analogous to salary) derived from monthly_spend
    (c.monthly_spend * 12) AS annual_spend,

    -- Spend / "salary" buckets
    CASE
        WHEN c.monthly_spend * 12 < 1000 THEN '<1k'
        WHEN c.monthly_spend * 12 < 5000 THEN '1k-5k'
        WHEN c.monthly_spend * 12 < 20000 THEN '5k-20k'
        ELSE '20k+'
        END                    AS annual_spend_bucket,

    -- Location bucket: country/region instead of exact city
    CASE
        WHEN c.city IN ('Berlin', 'Munich', 'Hamburg', 'Frankfurt') THEN 'DE'
        WHEN c.city IN ('London') THEN 'UK'
        WHEN c.city IN ('Paris') THEN 'FR'
       ...
        WHEN c.city IN ('Rome') THEN 'IT'
        WHEN c.city IN ('Stockholm') THEN 'SE'
        WHEN c.city IN ('Warsaw') THEN 'PL'
        WHEN c.city IN ('Cape Town') THEN 'ZA'
        WHEN c.city IN ('Singapore') THEN 'SG'
        ELSE 'OTHER'
        END                    AS location_bucket

    FROM customers_source AS c;
```

developer.confluent.io

CONFLUENT

Search

workspace-2025-03-03-080... | workspace-2025-02-18-0756... | workspace-2025-12-22-2037...

workspace-2025-12-22-203759

olena-env

Olena Kutsenko

User Account

AWS | us-east-1

AWS.us-east-1.env-j0jn2m.1ea6 | Running

Catalog olena-env Database cluster_0

```sql
SELECT
    customer_id,
    -- Annual spend (analogous to salary) derived from monthly_spend
    (c.monthly_spend * 12) AS annual_spend,

    -- Spend / “salary” buckets
    CASE
        WHEN c.monthly_spend * 12 < 1000 THEN '<1k'
        WHEN c.monthly_spend * 12 < 5000 THEN '1k–5k'
        WHEN c.monthly_spend * 12 < 20000 THEN '5k–20k'
        ELSE '20k+'
        END                AS annual_spend_bucket,

    -- Location bucket: country/region instead of exact city
    CASE
        WHEN c.city IN ('Berlin', 'Munich', 'Hamburg', 'Frankfurt') THEN 'DE'
        WHEN c.city IN ('London') THEN 'UK'
        WHEN c.city IN ('Paris') THEN 'FR'
        WHEN c.city IN ('San Francisco', 'New York') THEN 'US'
        WHEN c.city IN ('Bengaluru') THEN 'IN'
        WHEN c.city IN ('São Paulo') THEN 'BR'
        WHEN c.city IN ('Tokyo') THEN 'JP'
        WHEN c.city IN ('Sydney') THEN 'AU'
        WHEN c.city IN ('Toronto') THEN 'CA'
        WHEN c.city IN ('Amsterdam') THEN 'NL'
        WHEN c.city IN ('Barcelona') THEN 'ES'
        WHEN c.city IN ('Rome') THEN 'IT'
        WHEN c.city IN ('Stockholm') THEN 'SE'
```

Last saved at 2/9/26, 10:02 AM

7 / 20 cells

Adding noise



Adding tiny random changes to data to make individual values unreliable, but aggregates stay the same.

Name: Tom
Salary: 85000

→

Name: Tom
Salary: 87700

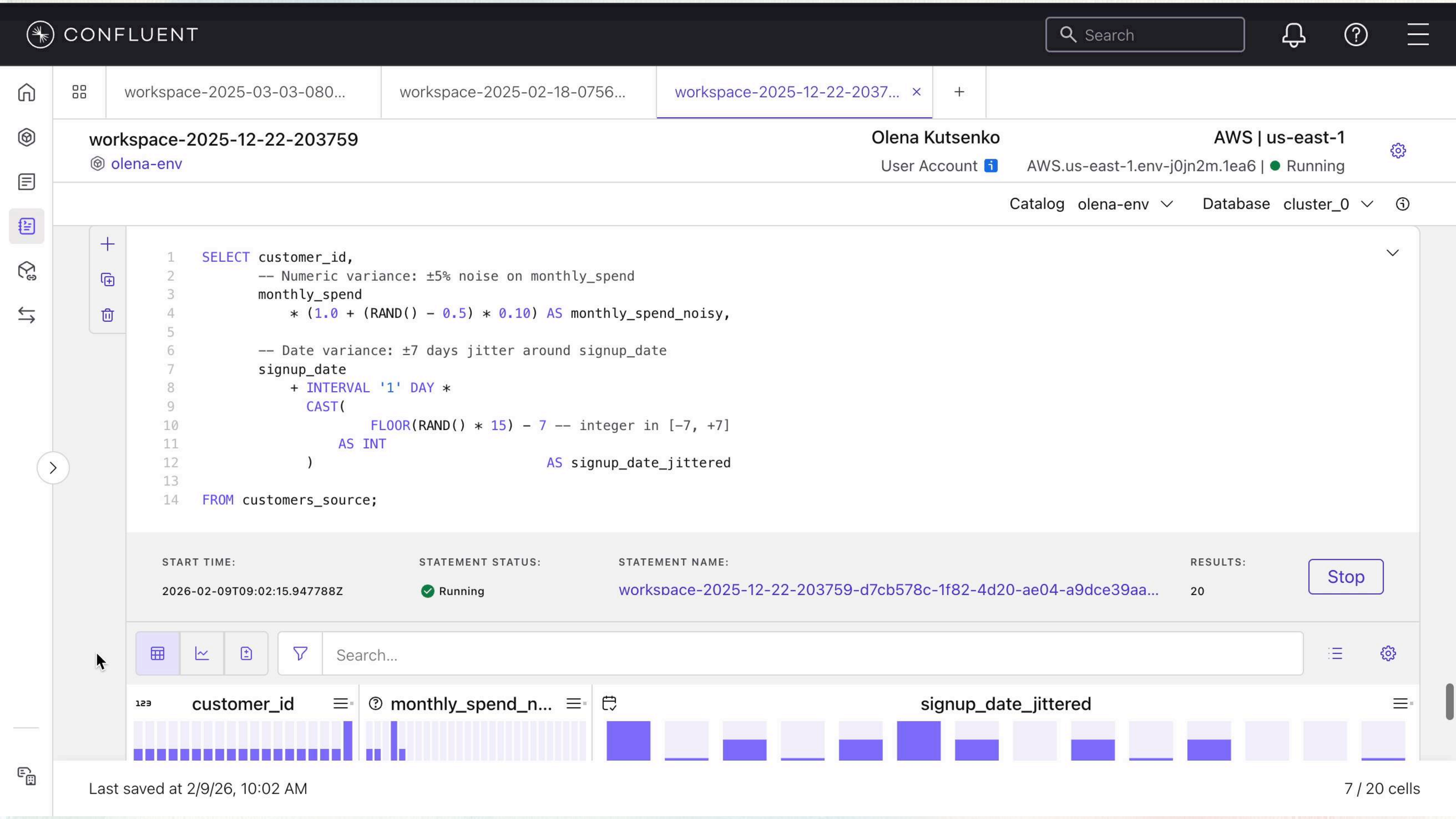
Methods

- Laplace
- Gaussian

```sql
SELECT customer_id,
       -- Numeric variance: ±5% noise on monthly_spend
       monthly_spend
           * (1.0 + (RAND() - 0.5) * 0.10) AS monthly_spend_noisy,

       -- Date variance: ±7 days jitter around signup_date
       signup_date
           + INTERVAL '1' DAY *
             CAST(
                     FLOOR(RAND() * 15) - 7 -- integer in [-7, +7]
                 AS INT
             )                                AS signup_date_jittered

FROM customers_source;
```


CONFLUENT
Search
workspace-2025-03-03-080...
workspace-2025-02-18-0756...
workspace-2025-12-22-2037...
workspace-2025-12-22-203759
olena-env
Olena Kutsenko
User Account
AWS | us-east-1
AWS.us-east-1.env-j0jn2m.1ea6 | Running
Catalog olena-env
Database cluster_0
1 SELECT customer_id,
2 -- Numeric variance: ±5% noise on monthly_spend
3 monthly_spend
4 * (1.0 + (RAND() - 0.5) * 0.10) AS monthly_spend_noisy,
5
6 -- Date variance: ±7 days jitter around signup_date
7 signup_date
8 + INTERVAL '1' DAY *
9 CAST(
10 FLOOR(RAND() * 15) - 7 -- integer in [-7, +7]
11 AS INT
12) AS signup_date_jittered
13
14 FROM customers_source;
START TIME:
2026-02-09T09:02:15.947788Z
STATEMENT STATUS:
Running
STATEMENT NAME:
workspace-2025-12-22-203759-d7cb578c-1f82-4d20-ae04-a9dce39aa...
RESULTS:
20
Stop
Search...
customer_id
monthly_spend_n...
signup_date_jittered
Last saved at 2/9/26, 10:02 AM
7 / 20 cells

Synthetic data generation



Strictly speaking data generation avoids the privacy problem entirely rather than solving it

Pros

- No direct identifiers
- No linkage attacks, no inference about real people
- Useful for sharing & development
- Often **far safer** than “anonymized” real data

Cons

- May reproduce near-identical individuals
- Poor for behavioral realism
- Deterministic and random

The privacy–utility trade-off

Rules of thumb

- Protect identifiers early
- Aggregate as soon as possible
- Store raw data only if you can justify it legally
- Measure data usefulness, not just privacy

Decision matrix

- High sensitivity + low value → drop it
- High sensitivity + high value → aggregate/tokenize
- Low sensitivity + high value → keep